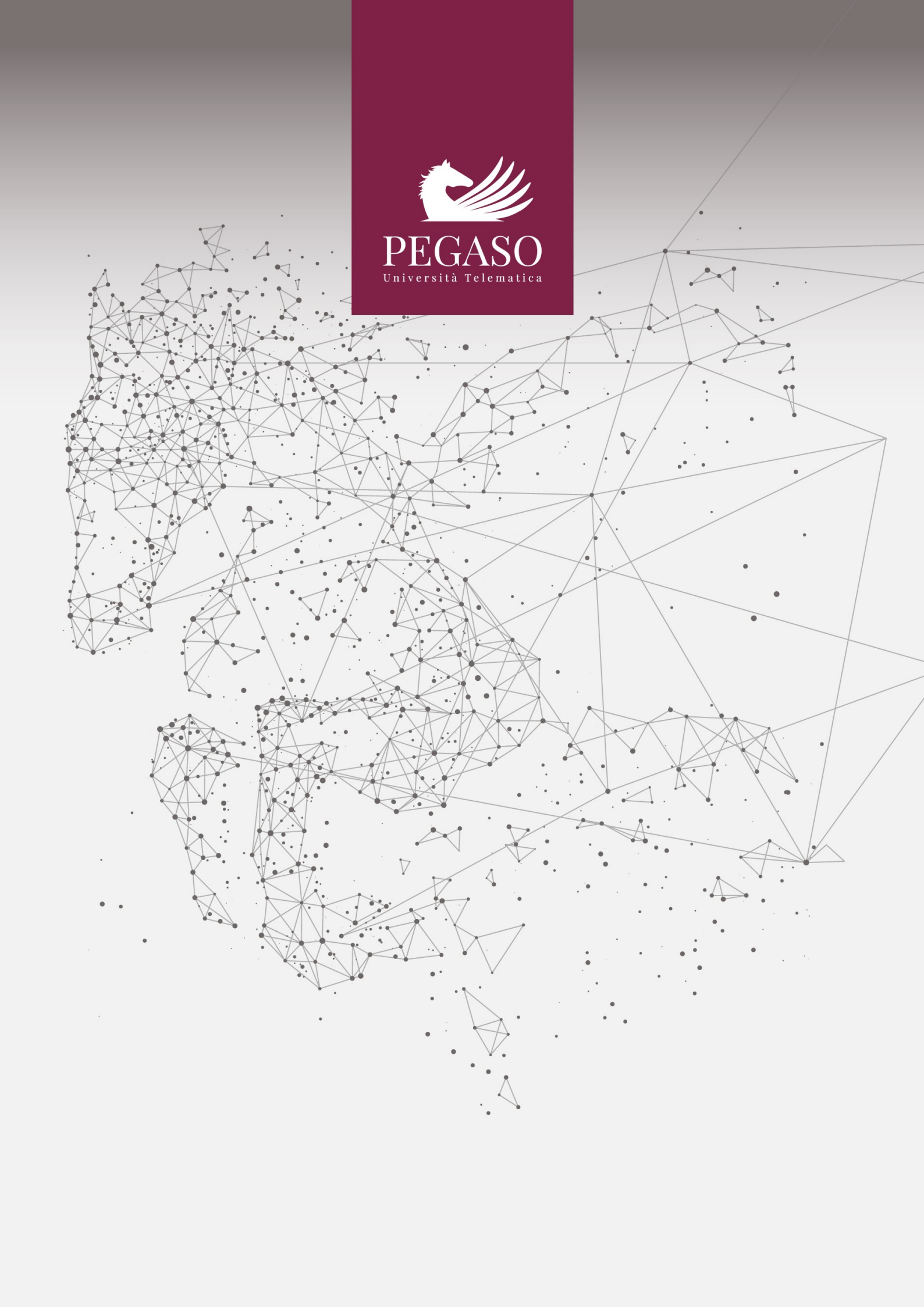

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. TRADUZIONE DELL'ISTRUZIONE IF-ELSE 3

2. TRADUZIONE DEL CICLO FOR 6

3. TRADUZIONE DEL CICLO WHILE 9

BIBLIOGRAFIA 12

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Traduzione dell'istruzione if-else

Tutti i linguaggi di programmazione ad Alto Livello mettono a disposizioni specifiche istruzioni per eseguire operazioni di base che consentono il controllo di flussi di esecuzione delle istruzioni diversi da quello puramente sequenziale.

L'struzione *if-else* permette l'esecuzione condizionale di due insiemi di istruzioni, in alternativa a seconda del valore di verità di una espressione logica specificata.

La forma sintattica dell'istruzione *if-else* nei Linguaggi C, C++ e Java è la seguente:

```
if (condizione logica) {
        insieme di istruzioni_1
}
else {
        insieme di istruzioni_2
}
```

le parentesi graffe sono facoltative se racchiudono singole istruzioni, altrimenti sono necessarie.

Se la valutazione della *condizione logica* ha valore *vero* si ha l'esecuzione dell'insieme di istruzioni_1, altrimenti se ha valore *falso* si ha l'esecuzione dell'insieme di istruzioni_2. Solo una delle due alternative viene eseguita perché la valutazione della *condizione logica* può fornire uno solo dei due valori di verità.

Il Linguaggio Assembly non include una istruzione specifica che svolga il controllo della esecuzione di insiemi di istruzioni in alternativa come l'istruzione *if-else*.

Questo risultato deve essere realizzato dal programmatore mediante un'opportuna combinazione delle istruzioni di base disponibili, in particolare, delle istruzioni di salto condizionato ed incondizionato.

Descriviamo qui di seguito un esempio di traduzione in Assembly MIPS dell'istruzione *if-else*. Per evidenziare la strutturazione della traduzione in Assembly MIPS consideriamo un esempio semplice dove si assume che tutte le variabili sono dichiarate di Tipo intero:

In fase di traduzione alle variabili vengono associati registri del processore. La seguente tabella ricorda gli indirizzi dei registri utilizzati per contenere dati, indicando la corrispondenza tra gli indirizzi numerici in Notazione decimale e la notazione simbolica utilizzata nell'Assembly MIPS:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

simbolo	*indirizzo*	*simbolo*	*indirizzo*	*simbolo*	*indirizzo*
$t0	8	$s0	16	$t8	24
$t1	9	$s1	17	$t9	25
$t2	10	$s2	18		
$t3	11	$s3	19		
$t4	12	$s4	20		
$t5	13	$s5	21		
$t6	14	$s6	22		
$t7	15	$s7	23		

la dichiarazione di variabili intere:

```
int  test, a, b, f  ;
```

Istruzioni in Linguaggio C:

```
if  ( test == 0 )  {
        f = a  ;
}
else  {
        f = b  ;
}
test = a–b  ;
```

salta *a ELSE* *se* test **==** 0 è ***falsa***

Come *condizione logica* dell'istruzione *if-else* si valuta se il valore della variabile test è uguale a 0.

Se la condizione è *vera* si effettua l'assegnamento f = a, se è *falsa* si effettua l'assegnamento f = b. In ogni caso l'istruzione successiva è l'assegnamento val = a-b.

Una traduzione in AssemblyMIPS con l'assegnazione delle variabili ai registri dell'Architettura MIPS

test	a	b	f
↓	↓	↓	↓
$t5	$t1	$t2	$s0.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

è data dalle istruzioni:

```
        bne   $t5,  $zero,  ELSE        #      salta a ELSE
                                        # se  test == 0  è  falsa,⟺
                                        # se  test != 0  è  vera
        addi  $s0,  $t1,  0             #  f = a
        j   END_IF                      #  esce da if-else
ELSE:   addi  $s0,  $t2,  0             #  f = b
END_IF: sub   $t5,  $t1,  $t2           #  test = a–b.
```

Si può notare che **nella traduzione Assembly** è stata utilizzata l'istruzione di salto condizionato ***bne*** che effettua il controllo sulla NEGAZIONE della *condizione logica* dell'*if-else*.

Questo consente di mantenere lo stesso ordine di presentazione delle istruzioni dell'*if-else*, con il caso *else* come secondo.

Inoltre è necessario introdurre una istruzione j di salto incondizionato per evitare l'esecuzione delle istruzioni relative al caso *else.*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Traduzione del ciclo for

Moltissime attività dei programmi richiedono l'esecuzione ripetuta di un insieme di istruzioni fino a quando una condizione risulta *vera*, ed i Linguaggi ad Alto Livello forniscono istruzioni che consentono di controllare questa esecuzione ciclica. Tra queste vi sono le istruzioni *for* e *while*.

Il ciclo *for* è un'istruzione che consente di ripetere l'esecuzione di un insieme di istruzioni per un fissato numero di volte.

La forma sintattica dell'istruzione *for* nei Linguaggi C, C++, Java, è la seguente:

for (*inizializzazione*; *condizione logica*; *incremento*) {

insieme di istruzioni

}

le parentesi graffe sono facoltative se racchiudono una singola istruzione, altrimenti sono necessarie.

L'esecuzione dell'istruzione *for*, come l'istruzione *while*, inizia effettuando la valutazione della *condizione logica.* Questo implica che, se la condizione risulta *falsa* l'esecuzione del ciclo termina senza eseguire l'insieme di istruzioni neppure una volta.

Le parentesi tonde contengono tre parti separate dal simbolo punto e virgola. L'*inizializzazione* viene effettuata solo la prima volta, prima di iniziare l'esecuzione dell'insieme di istruzioni. Al termine di tale esecuzione, viene effettuato l'*incremento* che tipicamente determina l'aggiornamento della *condizione logica*.

Quindi, si ripete la valutazione della *condizione logica* con l'esecuzione dell'insieme di istruzioni e dell'*incremento* fino a quando la condizione risulta *vera.*

Quando la valutazione della *condizione logica* risulta *falsa*, l'insieme di istruzioni non viene eseguito ed il ciclo *for* termina, passando all'esecuzione dell'istruzione successiva.

L'esecuzione di un ciclo *for* può anche determinare una ripetizione infinita dell'insieme di istruzioni se la *condizione logica* risulta sempre *vera*.

Il Linguaggio Assembly non include una istruzione specifica che consenta di ripetere l'esecuzione di un insieme di istruzioni per un fissato numero di volte come l'istruzione *for*.

Questo risultato deve essere realizzato dal programmatore mediante un'opportuna combinazione delle istruzioni di base disponibili, in particolare, delle istruzioni di salto condizionato ed incondizionato.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Descriviamo qui di seguito un esempio di traduzione in Assembly MIPS del ciclo *for*. Per evidenziare la strutturazione della traduzione in Assembly consideriamo un esempio semplice dove si assume che tutte le variabili sono dichiarate di Tipo intero.

ESEMPIO DI TRADUZIONE IN ASSEMBLY MIPS

Consideriamo la dichiarazione di variabili intere:

```
int  somma,  k,  Max  ;
```

e le istruzioni in Linguaggio C:

```
Max = 5  ;
for  ( k = 0 ;  k != Max ;  k = k+1 )  {
      somma = somma + k  ;
}
Max = 0  ;
```

uscita *dal ciclo for se* k **!=** Max *è* ***falsa***

L'*inizializzazione* pone a 0 la variabile k. La *condizione logica* del ciclo valuta se il valore della variabile k è diverso da quello della variabile Max. Se la condizione è *vera* si effettua l'assegnamento somma = somma + k, si esegue l'incremento k = k+1 che aggiorna la *condizione logica*, ed il ciclo di esecuzione viene ripetuto. Se la condizione è *falsa* si termina il ciclo *for* e si passa all'esecuzione dell'istruzione successiva Max = 0.

Una traduzione in Assembly MIPS con l'assegnazione delle variabili ai registri dell'Architettura MIPS

somma	k	Max
↓	↓	↓
$s1	$t0	$t5

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

è data dalle istruzioni:

```
        addi   $t5,  $zero,  5          # Max = 5
        add    $t0,  $zero,  $zero      # k = 0  inizializzazione
FOR:    beq    $t0,  $t5,  END_FOR      # uscita dal ciclo for
                                        # se k != Max è falsa, ⟺
                                        # se k == Max è vera
        add    $s1,  $s1,  $t0          # somma = somma + k
        addi   $t0,  $t0,  1            # k = k+1  incremento
        j    FOR                        # ripete il ciclo for
END_FOR: add   $t5,  $zero,  $zero      # Max = 0
```

Poiché l'USCITA dal ciclo *for* avviene se la *condizione logica* risulta FALSA, **nella traduzione Assembly** si utilizza una istruzione di salto condizionato che determina l'uscita dal ciclo quando la NEGAZIONE della *condizione logica* risulta VERA.

Nell'esempio la negazione della *condizione_logica* k != Max è data da k == Max, quindi si utilizza un salto condizionato su uguaglianza per uscire dal ciclo quando questa uguaglianza è *vera*.

Si nota anche che l'*inizializzazione* k = 0 è tradotta fuori dal ciclo *for*, la cui ripetizione viene realizzata mediante l'istruzione di salto incondizionato *j*.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Traduzione del ciclo while

L'istruzione *while* è un'istruzione che consente di controllare l'esecuzione ciclica di un insieme di istruzioni fino a quando una condizione risulta *vera*, e come l'istruzione *for* è presente nei Linguaggi ad Alto Livello.

La forma sintattica dell'istruzione *while* nei Linguaggi C, C++, Java, è la seguente:

```
while (condizione logica) {
        insieme di istruzioni
}
```

le parentesi graffe sono facoltative se racchiudono una singola istruzione, altrimenti sono necessarie.

L'esecuzione dell'istruzione *while*, come l'istruzione *for*, inizia effettuando la valutazione della *condizione logica.* Questo implica che, se la condizione risulta *falsa* l'esecuzione del ciclo termina senza eseguire l'insieme di istruzioni neppure una volta.

Le parentesi tonde contengono solo la *condizione logica* che controlla il ciclo. A differenza dell'istruzione *for*, il numero di ripetizioni del ciclo può non essere **esplicitamente** indicato.

All'inizio dell'esecuzione viene valutata la *condizione logica.* Se risulta *vera* si esegue l'insieme di istruzioni, dopo di che si valuta nuovamente la *condizione logica*. La ripetizione si interrompe quando la *condizione logica* risulta *falsa*. Questo determina l'uscita dal ciclo *while*, passando all'esecuzione dell'istruzione successiva.

L'esecuzione di un ciclo *while* può anche determinare una ripetizione infinita dell'insieme di istruzioni se la *condizione logica* risulta sempre *vera*.

Il Linguaggio Assembly non include una istruzione specifica che controlli l'esecuzione ciclica di un insieme di istruzioni fino a quando una condizione risulta *vera*, come l'istruzione *while*.

Questo risultato deve essere realizzato dal programmatore mediante un'opportuna combinazione delle istruzioni di base disponibili, in particolare, delle istruzioni di salto condizionato ed incondizionato.

Descriviamo qui di seguito un esempio di traduzione in Assembly MIPS dell'istruzione *while*. Per evidenziare la strutturazione della traduzione in Assembly consideriamo un esempio semplice dove si assume che tutte le variabili sono dichiarate di Tipo intero.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ESEMPIO DI TRADUZIONE IN ASSEMBLY MIPS

Consideriamo la dichiarazione di variabili intere:

```
int  val, conta  ;
```

e le istruzioni in Linguaggio C:

```
conta = 0  ;
while  ( val != 0 )  {
        val = val >>1 ;
        conta = conta + 1  ;
}
val = 7  ;
```

uscita *dal ciclo while*
se val **!=** 0 *è* ***falsa***

La *condizione logica* val != 0 è *vera* se il valore val è diverso da 0. In questo caso si esegue l'istruzione di shift a destra della sequenza binaria che rappresenta val, si incrementa di 1 la variabile conta, e si ripete la valutazione della *condizione logica* val != 0.

L'operazione di shift determina lo spostamento verso destra di una posizione di tutti i bit della sequenza, nell'ordine in cui si trovano, e assegna il valore 0 alla posizione che rimane vuota.

Poiché la sequenza è finita, dopo un numero finito di shift (che dipende dalla posizione del bit con valore 1 più a sinistra, se è presente) la sequenza sarà composta da tutti bit uguali a 0, determinando il valore *falso* nella valutazione della *condizione logica* val != 0. Questo causa l'interruzione del ciclo *while* ed il passaggio all'esecuzione dell'istruzione successiva val = 7.

Una traduzione in Assembly MIPS con l'assegnazione delle variabili ai registri dell'Architettura MIPS:

```
val          conta
 ↓            ↓
$s1          $t0
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

è data dalle istruzioni:

```
                add   $t0,  $zero,  $zero          #   conta = 0
WHILE:          beq  $s1,  $zero, END_WHILE   #uscita dal ciclo while
                                              #  se  val != 0   è falsa ⟺
                                              #  se  val == 0   è vera
                srl    $s1,  $s1,  1                #   val = val >>1
                addi  $t0,  $t0,  1                 #   conta = conta + 1
                j   WHILE                           #  ripete il ciclo while
END_WHILE:      addi $t5, $zero, 7                  #   val = 7
```

Poiché l'USCITA dal ciclo *while* avviene se la *condizione logica* risulta FALSA, **nella traduzione Assembly** si utilizza una istruzione di salto condizionato che determina l'uscita dal ciclo quando la NEGAZIONE della *condizione logica* risulta VERA.

Nell'esempio la negazione della *condizione_logica* val != 0 è data da val == 0, quindi si utilizza un salto condizionato su uguaglianza per uscire dal ciclo quando questa uguaglianza è *vera*.

La ripetizione dell'insieme di istruzioni viene realizzata mediante l'istruzione di salto incondizionato *j*.

OSSERVAZIONE

Nell'esempio si può anche notare che, a differenza del ciclo *for,* non c'è né l'inizializzazione di una variabile che controlla il numero di volte che il ciclo *while* viene eseguito, né un valore limite **esplicitamente** fissato per tali ripetizioni.

Le istruzioni *for* è *while* producono risultati del tutto equivalenti. La rispettiva strutturazione sintattica può rendere più adatto l'uso dell'una o dell'altra a seconda del tipo di controllo ciclico da realizzare.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- D. A. Patterson, J. L. Hennessy, "Struttura e progetto dei calcolatori", Zanichelli.
- S. B. Lippman, J. Lajoie, “C++ corso di programmazione”, Addison-Wesley.
- J. Lewis, W. Loftus, “Java. Fondamenti di progettazione software”, Addison-Wesley.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d’autore (L. 22.04.1941/n. 633).